

04_validation

March 27, 2026

1 04 — Validering

Validerer at koeffisientestimatene er robuste nok til å brukes i scenariosimulering. Spørsmålet er ikke *hvor godt modellen forutsier*, men **kan vi stole på $\hat{Q}$?*

Struktur: 1. Modellfit — sanitetsjekk (MAE/ R^2 , ikke kjerneargumentet) 2. Residualdiagnostikk (autokorrelasjon, heteroskedastisitet) 3. Funksjonell form (LOWESS residualer mot forbruk) 4. Koeffisientstabilitet (CV + årsvis) 5. Kryss-modell bekreftelse (XGBoost PDP) 6. Stratifisert fit etter sesong og etterspørsel

Input: intermediate/df_iso.parquet, intermediate/models.pkl,
intermediate/seasonal_medians.pkl, intermediate/cv_folds.pkl,
intermediate/preds_ols_seasonal.parquet, intermediate/preds_xgb.parquet,
intermediate/preds_msdr.parquet **Output:** intermediate/predictions.parquet

```
[60]: import os, pickle, warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.api as sm
from statsmodels.tsa.stattools import acf as compute_acf

from src.config import (
    TRAIN_YEARS, TEST_YEARS, INTERMEDIATE_DIR, FIG_DIR,
    DESEASON_TARGET, DESEASON_FEATURES, SEASON_ORDER,
    apply_style, C1, C2, C3, C4, C5, PALETTE
)
from src.evaluation import eval_metrics, demand_level

apply_style()
os.makedirs(f"{FIG_DIR}section_06", exist_ok=True)
```

1.1 Last data og prediksjoner fra alle modeller

```
[61]: df_iso = pd.read_parquet(f"{INTERMEDIATE_DIR}df_iso.parquet")

with open(f"{INTERMEDIATE_DIR}models.pkl", "rb") as f:
    models = pickle.load(f)

with open(f"{INTERMEDIATE_DIR}seasonal_medians.pkl", "rb") as f:
    med_data = pickle.load(f)
    medians = med_data["medians"]

with open(f"{INTERMEDIATE_DIR}cv_folds.pkl", "rb") as f:
    cv_data = pickle.load(f)

ols_model = models["ols_model"]
qr50_model = models["qr50_model"]
qr90_model = models["qr90_model"]

test_mask = df_iso.index.year.isin(TEST_YEARS)
df_test = df_iso[test_mask].copy()
print(f"Testperiode: {len(df_test):,} ISO-timer")
```

Testperiode: 9,844 ISO-timer

```
[62]: # Baseline-prediksjoner (NB03)
X_test = df_test[DESEASON_FEATURES]
X_test_c = sm.add_constant(X_test)
y_test = df_test[DESEASON_TARGET]

preds = pd.DataFrame(index=df_test.index)
preds["actual"] = y_test
preds["OLS"] = ols_model.predict(X_test_c)
preds["QR_Q50"] = qr50_model.predict(X_test_c)
preds["QR_Q90"] = qr90_model.predict(X_test_c)

# Lag-baselines
preds["Lag1"] = df_iso[DESEASON_TARGET].shift(1).reindex(preds.index)
preds["Lag24"] = df_iso[DESEASON_TARGET].shift(24).reindex(preds.index)

# Last prediksjoner fra nye modeller
new_models_loaded = []

try:
    p_seasonal = pd.read_parquet(f"{INTERMEDIATE_DIR}preds_ols_seasonal.
    ↪parquet")
    preds["OLS_seasonal"] = p_seasonal["OLS_seasonal"].reindex(preds.index)
    new_models_loaded.append("OLS_seasonal")
except FileNotFoundError:
```

```

print("OBS: preds_ols_seasonal.parquet ikke funnet - kjør NB03a først")

try:
    p_xgb = pd.read_parquet(f"{INTERMEDIATE_DIR}preds_xgb.parquet")
    preds["XGB"] = p_xgb["XGB"].reindex(preds.index)
    new_models_loaded.append("XGB")
except FileNotFoundError:
    print("OBS: preds_xgb.parquet ikke funnet - kjør NB03b først")

try:
    p_msdr = pd.read_parquet(f"{INTERMEDIATE_DIR}preds_msdr.parquet")
    preds["MSDR"] = p_msdr["MSDR"].reindex(preds.index)
    new_models_loaded.append("MSDR")
except FileNotFoundError:
    print("OBS: preds_msdr.parquet ikke funnet - kjør NB03c først")

# Metadata
preds["season"] = df_test["season"]
preds["cons_NO4"] = df_test["cons_NO4"]

train_cons = df_iso.loc[df_iso.index.year.isin(TRAIN_YEARS), "cons_NO4"]
t1, t2 = train_cons.quantile([1/3, 2/3])
preds["demand_level"] = preds["cons_NO4"].apply(lambda c: demand_level(c, t1,
↪t2))

print(f"\nLastet nye modeller: {new_models_loaded}")
print(f"Prediksjoner: {len(preds):,} rader, {len(preds.columns)} kolonner")

```

```

Lastet nye modeller: ['OLS_seasonal', 'XGB', 'MSDR']
Prediksjoner: 9,844 rader, 12 kolonner

```

1.2 Modellfit — sanitetsjekk

Overordnet MAE/R² bekrefter at modellene fanger prisvariasjonen rimelig godt. Dette er *ikke* kjerneargumentet for -robusthet — det er kun en sanitetsjekk på at modellen er fornuftig spesifisert.

```

[63]: ALL_MODEL_COLS = ["OLS", "OLS_seasonal", "QR_Q50", "QR_Q90", "XGB", "MSDR",
↪    "Lag1", "Lag24"]
ALL_MODEL_LABELS = {
    "OLS": "OLS (HAC)",
    "OLS_seasonal": "OLS (sesong)",
    "QR_Q50": "KR Q50",
    "QR_Q90": "KR Q90",
    "XGB": "XGBoost",
    "MSDR": "MS-DR (2 reg.)",

```

```

    "Lag1":          "Baseline lag-1",
    "Lag24":         "Baseline lag-24",
}

# Total evaluering
rows = []
for col in ALL_MODEL_COLS:
    if col not in preds.columns:
        continue
    m = eval_metrics(preds["actual"], preds[col])
    m["Modell"] = ALL_MODEL_LABELS.get(col, col)
    rows.append(m)

df_total = pd.DataFrame(rows)[["Modell", "MAE", "RMSE", "R2", "N"]]
print("=== Evaluering på testsettet (prisavvik) ===")
display(df_total)

```

=== Evaluering på testsettet (prisavvik) ===

	Modell	MAE	RMSE	R ²	N
0	OLS (HAC)	164.0	224.4	0.1971	9844
1	OLS (sesong)	212.5	275.6	-0.2115	9844
2	KR Q50	150.4	221.4	0.2184	9844
3	KR Q90	305.6	342.8	-0.8743	9844
4	XGBoost	177.1	251.0	-0.0049	9844
5	MS-DR (2 reg.)	186.0	238.4	0.0935	9844
6	Baseline lag-1	26.8	77.7	0.9037	9844
7	Baseline lag-24	94.9	197.9	0.3754	9844

1.3 Residualdiagnostikk (alle modeller)

```

[64]: # Residual-diagnostikk for strukturmodellene (ikke lag-baselines)
struct_models = [c for c in ["OLS", "OLS_seasonal", "XGB", "MSDR"]
                  if c in preds.columns]
n_models = len(struct_models)

fig, axes = plt.subplots(n_models, 3, figsize=(16, 4 * n_models))
if n_models == 1:
    axes = axes.reshape(1, -1)

colors = [C1, C2, C3, C4]

for i, col in enumerate(struct_models):
    resid = preds["actual"] - preds[col]
    label = ALL_MODEL_LABELS.get(col, col)
    c = colors[i % len(colors)]

```

```

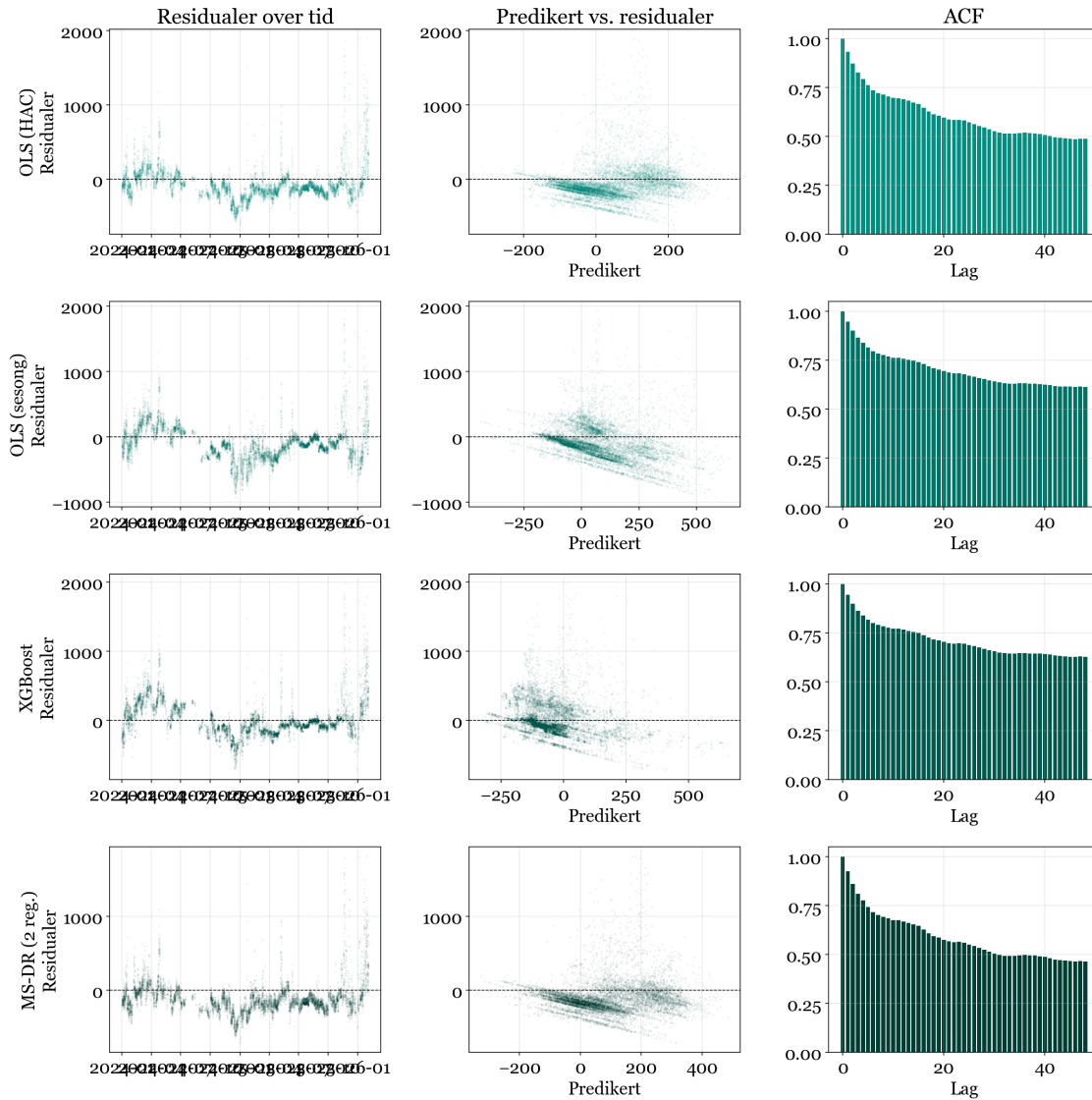
axes[i, 0].scatter(preds.index, resid, alpha=0.05, s=1, color=c)
axes[i, 0].axhline(0, color="black", lw=0.8, ls="--")
axes[i, 0].set_ylabel(f"{label}\nResidualer")
if i == 0:
    axes[i, 0].set_title("Residualer over tid")

axes[i, 1].scatter(preds[col], resid, alpha=0.05, s=1, color=c)
axes[i, 1].axhline(0, color="black", lw=0.8, ls="--")
axes[i, 1].set_xlabel("Predikert")
if i == 0:
    axes[i, 1].set_title("Predikert vs. residualer")

acf_vals = compute_acf(resid.dropna(), nlags=48, fft=True)
axes[i, 2].bar(range(len(acf_vals)), acf_vals, color=c, width=0.8)
axes[i, 2].set_xlabel("Lag")
if i == 0:
    axes[i, 2].set_title("ACF")

plt.tight_layout()
plt.savefig(f"{FIG_DIR}section_06/residualer_alle_modeller.pdf",
    ↪ bbox_inches="tight")
plt.show()

```



1.4 Funksjonell form — residualer mot forbruk (LOWESS)

Plott OLS-residualer mot forbruksvariabelen for å sjekke systematisk feilspesifikasjon av $Q \rightarrow P$ -relasjonen. En flat LOWESS-kurve bekrefter at en lineær spesifikasjon er tilstrekkelig.

```
[65]: from statsmodels.nonparametric.smoothers_lowess import lowess as sm_lowess

SEC08_DIR = f"{FIG_DIR}section_08/"
os.makedirs(SEC08_DIR, exist_ok=True)

_resid_models = [m for m in ["OLS", "OLS_seasonal"] if m in preds.columns]
```

```

_rlabels = {"OLS": "OLS (HAC)", "OLS_seasonal": "OLS (sesong)"}

cons_avvik_test = df_test["cons_avvik"].reindex(preds.index)

fig, axes = plt.subplots(1, len(_resid_models), figsize=(7 *
    ↪len(_resid_models), 5),
                        sharey=True)
if len(_resid_models) == 1:
    axes = [axes]

for ax, mcol in zip(axes, _resid_models):
    resid = (preds["actual"] - preds[mcol]).dropna()
    x      = cons_avvik_test.reindex(resid.index)
    valid = x.notna() & resid.notna()
    xv, rv = x[valid].values, resid[valid].values

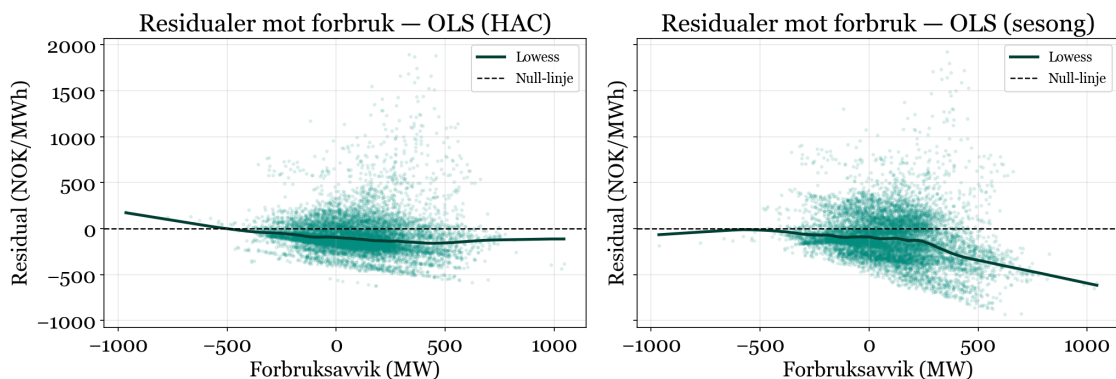
    ax.scatter(xv, rv, alpha=0.10, s=5, color=C1, rasterized=True)

    order = xv.argsort()
    smooth = sm_lowess(rv[order], xv[order], frac=0.15)
    ax.plot(smooth[:, 0], smooth[:, 1], color=C4, lw=2.5, label="Lowess")
    ax.axhline(0, color="black", lw=1.2, ls="--", label="Null-linje")

    ax.set_xlabel("Forbruksavvik (MW)")
    ax.set_ylabel("Residual (NOK/MWh)")
    ax.set_title(f"Residualer mot forbruk - {_rlabels[mcol]}")
    ax.legend(fontsize=12)

plt.tight_layout()
plt.savefig(f"{SEC08_DIR}resid_vs_cons.pdf", bbox_inches="tight")
plt.show()

```



1.5 Koeffisientstabilitet (CV, baseline OLS)

```
[66]: cv_df = cv_data["cv_df"]
      cv_coef_df = cv_data["cv_coef_df"]

      print("Rullerende CV (baseline OLS) - oppsummering:")
      print(f"  Antall folder: {len(cv_df)}")
      print(f"  MAE snitt:      {cv_df['MAE'].mean():.1f}")
      print(f"  MAE std:         {cv_df['MAE'].std():.1f}")
      print(f"\n  cons_avvik beta snitt: {cv_coef_df['cons_avvik'].mean():.4f}")
      print(f"  cons_avvik beta std:  {cv_coef_df['cons_avvik'].std():.4f}")

      fig, axes = plt.subplots(1, 2, figsize=(14, 5))

      axes[0].plot(cv_coef_df["test_q"], cv_coef_df["cons_avvik"],
                   marker="o", color=C1, ms=4)
      axes[0].axhline(ols_model.params["cons_avvik"], color=C4, ls="--",
                      label=f"Full modell: {ols_model.params['cons_avvik']:.4f}")
      axes[0].set_ylabel("beta (cons_avvik)")
      axes[0].set_title("Forbrukskoeffisient over CV-folder")
      axes[0].legend()
      axes[0].tick_params(axis="x", rotation=45)

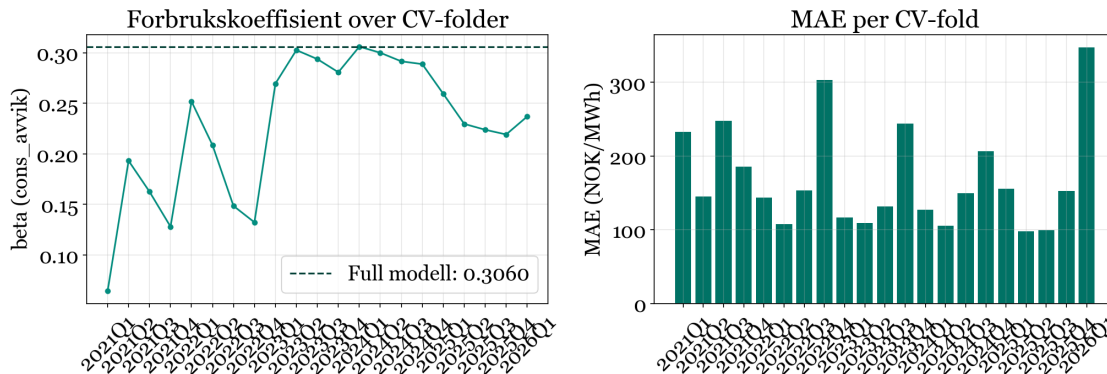
      axes[1].bar(cv_df["test_q"], cv_df["MAE"], color=C2)
      axes[1].set_ylabel("MAE (NOK/MWh)")
      axes[1].set_title("MAE per CV-fold")
      axes[1].tick_params(axis="x", rotation=45)

      plt.tight_layout()
      plt.savefig(f"{FIG_DIR}section_06/cv_stabilitet.pdf", bbox_inches="tight")
      plt.show()
```

Rullerende CV (baseline OLS) - oppsummering:

Antall folder: 21
MAE snitt: 169.7
MAE std: 69.7

cons_avvik beta snitt: 0.2281
cons_avvik beta std: 0.0683



1.6 Stabilitetssjekk $\hat{Q}$ per år

Estimer OLS (HAC) og OLS (sesong) separat for 2023, 2024 og 2025. Plott forbrukskoeffisienten med 95 % HAC-konfidensintervall per år. Stabile estimater på tvers av år er nødvendig for at ΔP -beregningene skal ha ekstern validitet.

```
[67]: import os
SEC07_DIR = f"{FIG_DIR}section_07/"
os.makedirs(SEC07_DIR, exist_ok=True)

STAB_YEARS = [2023, 2024, 2025]
_stab_rows = []

for yr in STAB_YEARS:
    df_yr = df_iso[df_iso.index.year == yr].copy()
    y_yr = df_yr[DESEASON_TARGET]
    X_yr = df_yr[DESEASON_FEATURES]
    res_o = sm.OLS(y_yr, sm.add_constant(X_yr)).fit(
        cov_type="HAC", cov_kws={"maxlags": 24}
    )
    ci_o = res_o.conf_int(alpha=0.05)
    _stab_rows.append({
        "År": yr,
        "r": res_o.params["cons_avvik"],
        "ci_lo": ci_o.loc["cons_avvik", 0],
        "ci_hi": ci_o.loc["cons_avvik", 1],
    })

stab_df = pd.DataFrame(_stab_rows)
print(f"Referanse: OLS (HAC) full modell  $\hat{Q}$  = {ols_model.params['cons_avvik']:.5f}")
display(stab_df.set_index("År").round(5))
```

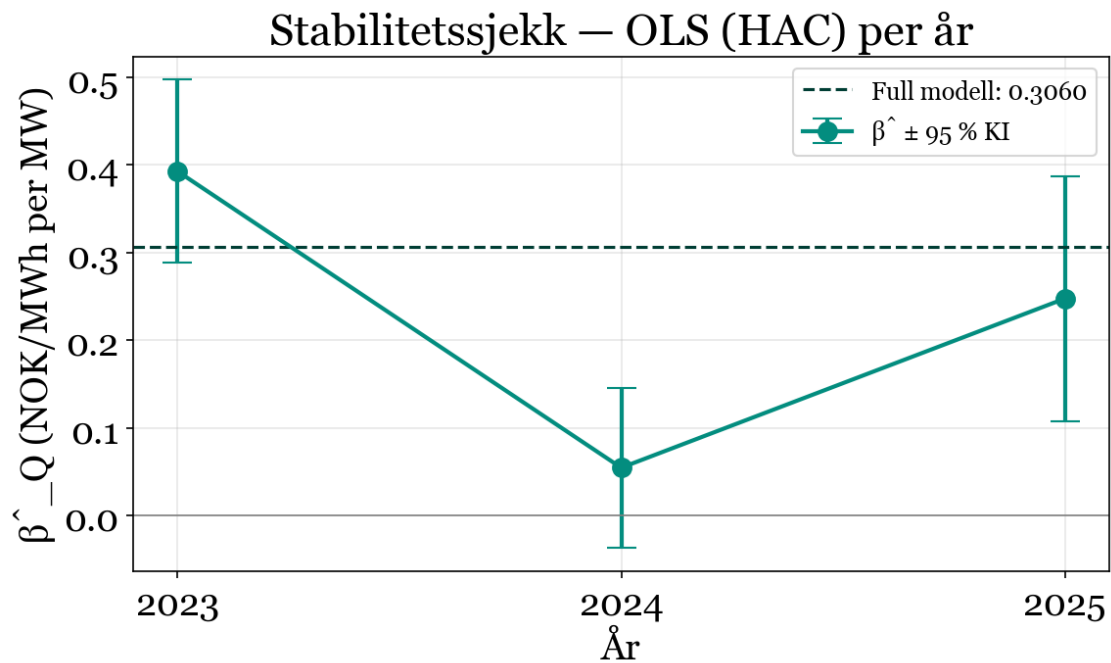
```

fig, ax = plt.subplots(figsize=(8, 5))
ax.errorbar(
    stab_df["År"], stab_df["^"],
    yerr=[stab_df["ci_lo"] - stab_df["^"], stab_df["ci_hi"] - stab_df["^"]],
    fmt="o-", color=C1, capsize=7, ms=9, lw=2, label="^ ± 95 % KI"
)
ax.axhline(ols_model.params["cons_avvik"], color=C4, ls="--", lw=1.5,
            label=f"Full modell: {ols_model.params['cons_avvik']:.4f}")
ax.axhline(0, color="grey", lw=0.8)
ax.set_xticks(STAB_YEARS)
ax.set_xlabel("År")
ax.set_ylabel("^_Q (NOK/MWh per MW)")
ax.set_title("Stabilitetssjekk - OLS (HAC) per år")
ax.legend(fontsize=12)
plt.tight_layout()
plt.savefig(f"{SEC07_DIR}stab_beta_Q.pdf", bbox_inches="tight")
plt.show()

```

Referanse: OLS (HAC) full modell $\hat{\beta}_Q = 0.30600$

	$\hat{\beta}$	ci_lo	ci_hi
År			
2023	0.39268	0.28829	0.49707
2024	0.05483	-0.03656	0.14622
2025	0.24765	0.10789	0.38741



1.7 Kryss-modell bekreftelse — Partial Dependence Plot (XGBoost)

PDP for forbruksvariabelen. Det lokale stigningstallet i PDP rundt observert forbruksnivå er analogt med $\hat{Q}$ i OLS — og bekrefter at den lineære effekten ikke er et artefakt av OLS-spesifikasjonen.

```
[68]: SEC09_DIR = f"{FIG_DIR}section_09/"
os.makedirs(SEC09_DIR, exist_ok=True)

# Last XGBoost-modell
with open(f"{INTERMEDIATE_DIR}models_xgb.pkl", "rb") as f:
    _xgb_data_04 = pickle.load(f)
_xgb_model_04 = _xgb_data_04["model"]
_XGB_FEATURES = _xgb_data_04["features"] #_
    ↪ ['cons_avvik', 'fill_avvik', 'wind_c', 'season_code']

_season_map_04 = {"Winter": 0, "Spring": 1, "Summer": 2, "Autumn": 3}

X_xgb_all = df_iso[DESEASON_FEATURES].copy()
X_xgb_all["season_code"] = df_iso["season"].map(_season_map_04)
X_xgb_all = X_xgb_all[_XGB_FEATURES]

# Manuel PDP: sett cons_avvik til hvert punkt i gitteret, hold resten observert
_grid = np.linspace(
    X_xgb_all["cons_avvik"].quantile(0.05),
    X_xgb_all["cons_avvik"].quantile(0.95),
    100
)
_pdp = []
for _c in _grid:
    _Xtmp = X_xgb_all.copy()
    _Xtmp["cons_avvik"] = _c
    _pdp.append(_xgb_model_04.predict(_Xtmp).mean())
_pdp = np.array(_pdp)

# Lokalt stigningstall rundt mediannivå (±1 std)
_mu = X_xgb_all["cons_avvik"].mean()
_sd = X_xgb_all["cons_avvik"].std()
_msk = (_grid >= _mu - _sd) & (_grid <= _mu + _sd)
_slope_pdp, _int_pdp = np.polyfit(_grid[_msk], _pdp[_msk], 1)
_pdp_dp_230 = _slope_pdp * 230

print(f"PDP lokalt stigningstall:  {_slope_pdp:.5f} NOK/MWh per MW")
print(f"Estimert ΔP (+230 MW):      {_pdp_dp_230:.1f} NOK/MWh")
print(f"OLS (HAC) referanse:         {ols_model.params['cons_avvik'] * 230:.1f}_"
    ↪ "NOK/MWh")
```

```

_mid_idx = int(np.abs(_grid - _mu).argmin())
_tang_x = np.array([_mu - _sd, _mu + _sd])
_tang_y = _slope_pdp * (_tang_x - _mu) + _pdp[_mid_idx]

fig, ax = plt.subplots(figsize=(9, 5))
ax.plot(_grid, _pdp, color=C1, lw=2.5, label="PDP (XGBoost)")
ax.axvspan(_mu - _sd, _mu + _sd, alpha=0.10, color=C2,
           label="±1 std rundt median")
ax.axvline(_mu, color=C4, ls=":", lw=1.5, label="Median forbruk")
ax.plot(_tang_x, _tang_y, color=C3, lw=2, ls="--",
        label=f"Lokal tangent: {_slope_pdp:.4f} NOK/MWh per MW")

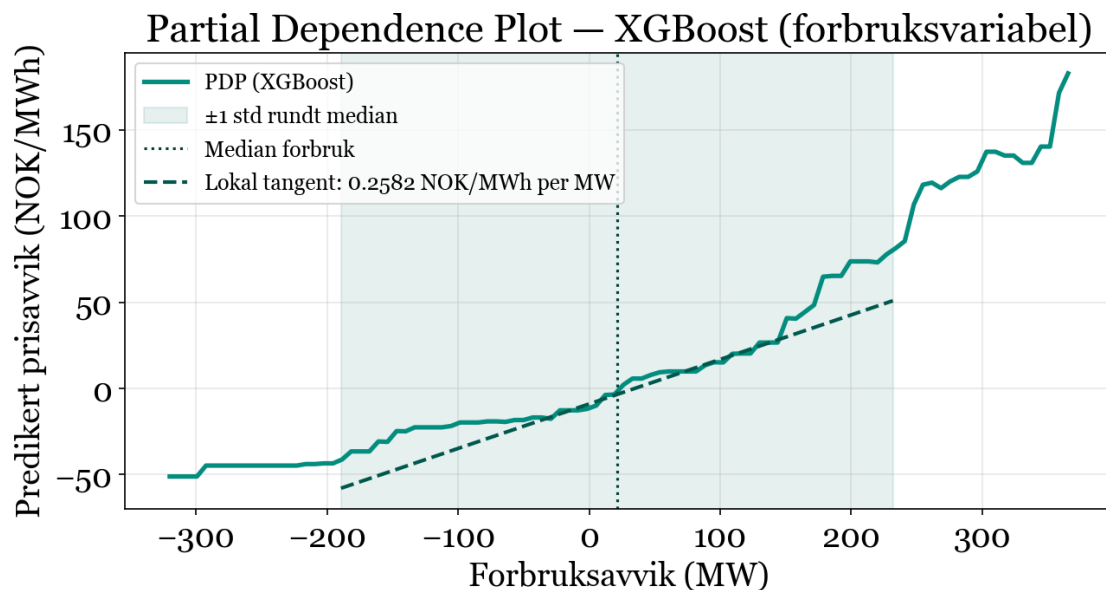
ax.set_xlabel("Forbruksavvik (MW)")
ax.set_ylabel("Predikert prisavvik (NOK/MWh)")
ax.set_title("Partial Dependence Plot - XGBoost (forbruksvariabel)")
ax.legend(fontsize=12)
plt.tight_layout()
plt.savefig(f"{SEC09_DIR}pdp_xgb.pdf", bbox_inches="tight")
plt.show()

```

PDP lokalt stigningstall: 0.25821 NOK/MWh per MW

Estimert ΔP (+230 MW): 59.4 NOK/MWh

OLS (HAC) referanse: 70.4 NOK/MWh



1.8 Stratifisert modellfit

MAE per sesong og etterspørselsnivå. Viser hvordan modellene presterer ulikt under ulike markedsforhold — nyttig bakgrunnsinformasjon, ikke kjernevalidering.

1.9 Per sesong

```
[69]: season_rows = []
for s_en, s_no in SEASON_ORDER.items():
    mask = preds["season"] == s_en
    sub = preds[mask]
    if len(sub) < 10:
        continue
    for col in ALL_MODEL_COLS:
        if col not in sub.columns:
            continue
        m = eval_metrics(sub["actual"], sub[col])
        m["Modell"] = ALL_MODEL_LABELS.get(col, col)
        m["Sesong"] = s_no
        season_rows.append(m)

df_season = pd.DataFrame(season_rows)
print("=== MAE per sesong ===")
pivot = df_season.pivot(index="Modell", columns="Sesong", values="MAE")
display(pivot[["Vinter", "Vår", "Sommer", "Høst"]])
```

=== MAE per sesong ===

Sesong	Vinter	Vår	Sommer	Høst
Modell				
Baseline lag-1	37.4	30.2	13.5	20.5
Baseline lag-24	134.3	89.7	43.9	91.2
KR Q50	205.7	144.0	93.9	130.1
KR Q90	355.6	219.2	287.8	342.5
MS-DR (2 reg.)	231.3	143.2	154.3	195.9
OLS (HAC)	214.4	145.6	113.4	158.1
OLS (sesong)	284.6	255.5	87.8	179.1
XGBoost	220.6	237.5	87.9	132.8

1.10 Per etterspørselsnivå

```
[70]: demand_rows = []
for dl in ["Lav", "Normal", "Høy"]:
    mask = preds["demand_level"] == dl
    sub = preds[mask]
    if len(sub) < 10:
```

```

        continue
    for col in ALL_MODEL_COLS:
        if col not in sub.columns:
            continue
        m = eval_metrics(sub["actual"], sub[col])
        m["Modell"] = ALL_MODEL_LABELS.get(col, col)
        m["Etterspørsel"] = dl
        demand_rows.append(m)

df_demand = pd.DataFrame(demand_rows)
print("=== MAE per etterspørselsnivå ===")
pivot_d = df_demand.pivot(index="Modell", columns="Etterspørsel", values="MAE")
display(pivot_d[["Lav", "Normal", "Høy"]])

```

=== MAE per etterspørselsnivå ===

Etterspørsel	Lav	Normal	Høy
Modell			
Baseline lag-1	13.5	20.3	37.0
Baseline lag-24	41.8	67.0	136.2
KR Q50	93.2	122.5	193.6
KR Q90	260.2	290.3	335.7
MS-DR (2 reg.)	141.9	157.9	223.3
OLS (HAC)	108.3	134.0	207.8
OLS (sesong)	102.7	183.0	280.8
XGBoost	89.3	158.6	228.5

1.11 Lagre samlet prediksjonsdata

```

[71]: preds.to_parquet(f"{INTERMEDIATE_DIR}predictions.parquet")
print(f"Lagret: {INTERMEDIATE_DIR}predictions.parquet ({len(preds):,} rader)")

```

Lagret: intermediate/predictions.parquet (9,844 rader)